



4.4 EXAMPLE 2

MIKROELEKTRONIKA

Using assembly instructions and internal oscillator LFINTOSC...

This is actually a sequel to the previous example, but deals with a bit more complicated problem... The idea is to make LED diodes on the port B blink slowly. It can be done by setting large value for delay parameter in the *Delay* function. But there is also another, more efficient manner to make this happen. You remember that this microcontroller has built-in oscillator LFINTOSC which operates at the frequency of 31kHz? Now, it's time to 'give it a chance'.

The program starts with the *do-while* loop and remains there for 20 cycles. After each iteration, 100mS delay is provided, which is reflected as relatively fast LED blinking of the port B. When the program exits this loop, the microcontroller starts using the LFINTOSC oscillator as a clock signal source. The LED blinking is considerably slower now even though the program executes the same *do-while* loop with 10 times shorter delay.

For the purpose of making some potentially dangerous situation more obvious here, control bits are activated by assembly instructions. Simply put, when entering or exiting the assembly instruction in the program, the compiler doesn't save data on currently active RAM bank, which means that in this program section, bank selection depends on the SFR registers in use. When switching back to the program section written in C, the control bits RP0 and RP1 must return the state they had before 'assembly language adventure'. In this program, the problem is solved by using the *saveBank* auxiliary variable which saves the state of these two bits.

```

1  /* Header *****/
2
3  int k = 0;
4  char saveBank;
5
6  void main() {
7      ANSEL = 0;           // All I/O pins are configured as digital
8      ANSELH = 0;
9      PORTB = 0;          // All port B pins are set to 0
10     TRISB = 0;           // Port B pins are configured as outputs
11
12     do {
13         PORTB = ~PORTB;   // Invert port B logic state
14         Delay_ms(100);    // 100mS delay
15         k++;              // Increment k by 1
16     }
17     while(k<20);          // Remain in loop while k<20
18
19     k=0;                  // Reset variable k
20     saveBank = STATUS & 0b01100000; // Save the state of bits RP0 and RP1
21                                     // (bits 5 and 6 of the STATUS register)
22
23     asm {                 // Start of assembly sequence
24         bsf STATUS,RP0    // Select memory bank containing the OSCCON
25         bcf STATUS,RP1    // register
26         bcf OSCCON,6      // Select internal oscillator LFINTOSC
27         bcf OSCCON,5      // of 31kHz frequency
28         bcf OSCCON,4
29         bsf OSCCON,0      // Microcontroller uses internal oscillator
30     }                     // End of assembly sequence
31
32     STATUS &= 0b10011111; // Bits RP0 and RP1 return their original state
33     STATUS |= saveBank;
34
35     do {
36         PORTB = ~PORTB;   // Invert port B logic state

```

```
37     Delay_ms(10);           // 10 mS delay
38     k++;                     // Increment k by 1
39 }
40 while(k<20);                 // Remain in loop while k<20
41 }
```

You have noticed that the clock signal source is changed ‘on the fly’. If you want to make sure of it, remove quartz crystal prior to switching the microcontroller on. The microcontroller will not start to operate because the *Config Word* loaded with the program requires the use of crystal on switching on. If you remove the crystal later during operation, nothing will happen, it will not affect the microcontroller at all.



4.4 Example 2 by MikroElektronika is licensed under a Creative Commons Attribution 4.0 International License, except where otherwise noted.